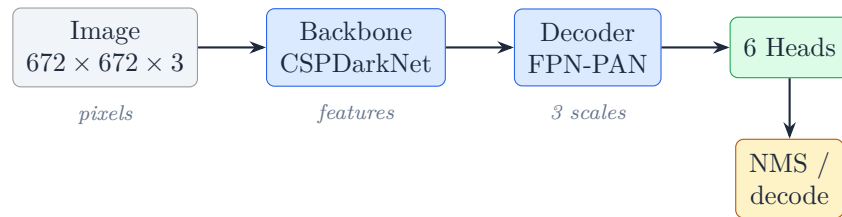


An Illustrated Guide

to the YOLOv8 Polygon + Distance codebase

A TensorFlow reimplementation — explained from scratch



Detection • Polygon segmentation • Distance estimation

39 classes | 6 output heads | PolyYOLO 24-vertex radial polygons | per-object distance

This guide is generated from the project's own code and docs. Every “real-pipeline” figure is produced by running the actual augmentation / model code (see [docs/codebase_guide/gen_figures.py](#)). Edit [chapters/*.tex](#) and rebuild to change it.

Contents

How to read this guide	3
1 Foundations: the ideas behind object detection	5
1.1 What “object detection” actually asks for	5
1.2 Bounding boxes and coordinate conventions	5
1.3 Intersection over Union (IoU)	6
1.4 Confidence scores and NMS	6
1.5 The grid idea: how YOLO predicts	6
1.6 Feature pyramids and multiple scales	6
1.7 Anchor-free detection	6
1.8 Two ways to describe an object’s shape	7
1.9 Estimating distance	7
1.10 How a model “learns where things are”	7
2 The system at a glance	8
2.1 The end-to-end path	8
2.2 Three tiers, one codebase	8
2.3 The six heads	9
2.4 What “training” produces	9
3 The data pipeline	11
3.1 The stage order	11
3.2 Sampling from several datasets	12
3.3 Copy-Paste, then Mosaic	12
3.3.1 Mosaic in depth	13
3.4 Flip and color	14
3.5 Merging the distance dataset	15
4 Polygons: the PolyYOLO radial representation	16
4.1 The radial idea	16
4.2 Why confidence sees every bin	17
4.3 Arc-length resampling: making bins track real shapes	17
4.4 Three formats across the stack	17
4.5 Decoding a predicted polygon	18
5 The model architecture	20
5.1 Backbone: turning pixels into features	20
5.2 Decoder: the FPN-PAN	20
5.3 The six heads	21
5.4 How the box head works: DFL	21
5.5 Anchors and strides	21
5.6 Inference: the detection generator	22

6	The losses	23
6.1	First, the assignment problem	23
6.2	The loss tree	24
6.2.1	Box: CIoU and DFL	24
6.2.2	Classification	24
6.2.3	Polygon: three sub-losses	24
6.2.4	Distance	24
6.3	Gains and normalization	24
7	Training: optimizer, schedules, and the loop	26
7.1	The optimizer: SGD with warmup	26
7.2	EMA: a smoothed copy of the weights	27
7.3	The training loop and epoch accounting	27
7.4	Mixed precision and XLA	27
7.5	Multiple GPUs	28
7.6	What you can watch while it trains	28
8	Configuration: how a run is described	29
8.1	YAML to dataclasses	29
8.1.1	Inheritance	29
8.2	The structure	29
8.3	The knobs you will touch most	30
8.4	Derived fields — don't hand-edit	30
8.5	Validated invariants	30
9	Evaluation: measuring quality	31
9.1	Detection metrics (COCO)	31
9.2	Polygon metrics	31
9.3	Distance metrics	32
9.4	Running it	32
10	Tools, testing, and on-device export	33
10.1	The scripts you run	33
10.2	Checkpoint migration and warm-starting	33
10.3	Testing	34
10.4	On-device export (Qualcomm SNPE / DLC)	34
11	Appendix	35
11.1	Glossary	35
11.2	Where everything lives	36
11.3	Design decisions worth remembering	36

How to read this guide

Welcome! This guide explains an entire deep-learning codebase — a TensorFlow reimplementation of **YOLOv8** that also predicts **object outlines** (polygons) and **how far away** each object is. It is written for someone who is comfortable with Python but is *new* to object detection. We start from the ideas, then walk through the actual code, file by file.

Who this is for

You will get the most out of this if you know basic Python and a little about neural networks (“a model takes an input, has weights, and is trained with gradient descent”). You do *not* need prior knowledge of object detection, anchors, IoU, segmentation, or YOLO. Every such term is introduced when first used.

The four kinds of callout boxes

Throughout the guide you will see colored boxes. Each has a job:

> Beginner’s note

Explains a concept from scratch — what it means and why it exists. Read these if a term is new to you.

Under the hood

Shows the concrete mechanism in *this* codebase: the function, the tensor shape, the exact formula. Read these to connect the idea to the code.

Why this codebase does it this way

A non-obvious design decision and the reasoning behind it. Several choices here differ from “stock” YOLOv8 on purpose; these boxes say why.

Gotcha

A subtle trap — a coordinate-order mismatch, a sentinel value, a normalization convention — that causes real bugs if you get it wrong.

Conventions

File and symbol names look like `data_pipeline/mosaic.py` and `Mosaic._mosaic`. Tensor shapes are written `[N, 72]`. Where coordinates matter we always say which convention is in play — `yxyx` vs `xyxy`, normalized `[0,1]` vs pixels — because mixing them up is the single most common source of bugs in this kind of code.

A map of the journey

1. **Foundations** — the object-detection ideas you need (boxes, IoU, NMS, anchors, feature pyramids, segmentation, the YOLO idea).
2. **The system at a glance** — the three model tiers and the six heads.
3. **The data pipeline** — how images and labels are loaded and augmented.
4. **Polygons** — the radial “PolyYOLO” representation, in depth.
5. **The model** — backbone, decoder, heads, and post-processing.
6. **The losses** — task-aligned assignment and every loss term.
7. **Training** — optimizer, schedules, EMA, the training loop.
8. **Configuration, evaluation, tooling** — how to actually run it.

Let’s begin.

Chapter 1

Foundations: the ideas behind object detection

This chapter builds the vocabulary. If you already know what IoU, anchors, NMS, feature pyramids, and segmentation are, you can skim. Otherwise, read it once and the code chapters will read much more easily.

1.1 What “object detection” actually asks for

Image *classification* answers “what is in this picture?” with a single label. **Object detection** is harder: it must answer “*what* objects are here, and *where* is each one?” — producing, for every object, a **bounding box** (a rectangle) and a **class label**. This codebase then adds two more questions per object: “what is its exact outline?” and “how far away is it?”.

1.2 Bounding boxes and coordinate conventions

A box is four numbers. But there are several ways to write those four numbers, and mixing them up is the #1 bug in detection code.

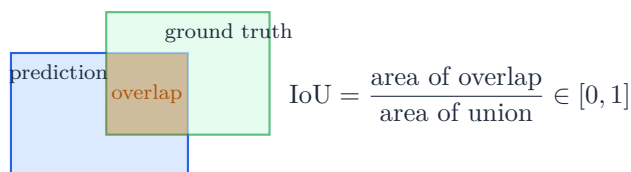
Convention	The four numbers
xyxy	left, top, right, bottom (two corners), x first
yxyx	top, left, bottom, right (two corners), y first
xywh	center-x, center-y, width, height
normalized	each value divided by image width/height, so in $[0, 1]$
pixels	raw pixel coordinates

Gotcha

This repo deliberately uses **two** conventions and converts between them at a known seam. Ground-truth boxes coming out of the dataset decoders and parsers are **yxyx, normalized** to $[0, 1]$. The loss and the label-assignment code convert to **xyxy in pixels**. Whenever you touch box math, first ask “which convention am I in?”. The codebase is careful to label this everywhere; you should be too.

1.3 Intersection over Union (IoU)

To say whether a predicted box is “correct”, we need to measure how well it overlaps a ground-truth box. The standard measure is **Intersection over Union**: the area of overlap divided by the area of their union.



IoU is 1 for a perfect match and 0 for no overlap. A common rule is “a detection counts as correct if its IoU with a true box is at least 0.5”. IoU appears everywhere: in the metric that scores the model, in NMS, and — in a refined form called **CIoU** — inside the box loss.

1.4 Confidence scores and NMS

Each detection comes with a **confidence** (or score) in $[0, 1]$: how sure the model is. A detector fires on the same object from several adjacent grid cells, so we get a cluster of overlapping boxes. **Non-maximum suppression** cleans this up: sort by score, keep the top box, delete every other box that overlaps it too much (IoU above a threshold), repeat.

Under the hood

Here NMS runs in `models/detection_generator.py` with `max_boxes=300`, `nms_thresh=0.65`, `score_thresh=0.05`, and it is **per-class**: classes do not suppress each other, so an overlapping “person” and “bag” both survive.

1.5 The grid idea: how YOLO predicts

YOLO (“You Only Look Once”) does not propose-then-classify. It overlays a **grid** on the feature map and asks *every cell* to predict whether an object is centered there, and if so its box, class, and (here) polygon and distance. One forward pass yields all predictions — fast and simple.

> Beginner’s note

Stride. A backbone shrinks the image. If a feature map is $8\times$ smaller than the input, we say it has **stride 8**: each cell “sees” an 8×8 pixel patch. Small strides (fine grids) catch small objects; large strides (coarse grids) catch large ones.

1.6 Feature pyramids and multiple scales

A single grid can’t handle a tiny sign and a huge truck equally well. So detectors use a **feature pyramid**: several feature maps at different strides. This codebase uses three, at strides **8, 16, and 32** (grids of 84×84 , 42×42 , 21×21 for a 672-pixel input). Every head predicts at all three. The decoder that builds and fuses these maps is an **FPN-PAN** — covered in the architecture chapter.

1.7 Anchor-free detection

Older detectors placed several preset box shapes (“anchors”) at each cell and predicted offsets from them. YOLOv8 is **anchor-free**: each cell has a single *anchor point* at its center, and the

box head predicts the distances from that point to the four sides. The anchor points are simply the cell centers, $(i + 0.5) \cdot \text{stride}$, built inside the loss and the detection generator.

1.8 Two ways to describe an object’s shape

A bounding box is a crude outline. **Segmentation** describes the actual shape. There are two common encodings:

- a **mask** — a per-pixel “inside/outside” map (heavy, but exact);
- a **polygon** — an ordered list of boundary points (compact).

This codebase uses a special *radial* polygon format from “PolyYOLO”: 24 vertices arranged at fixed 15° angles around the box center. It is light enough to predict with three small heads and is the subject of Chapter 4.

1.9 Estimating distance

The last extra task is **monocular distance estimation**: from a single RGB image, predict how far each object is, in meters. This is learned from a separate labeled dataset and predicted by a one-channel head on a log scale. It is trained jointly with detection, merged at the batch level — a neat trick we will see in the data-pipeline chapter.

1.10 How a model “learns where things are”

One subtlety underlies all of detection training. The model makes predictions at thousands of grid cells, but a training image has only a handful of objects. Which cells should be held responsible for which object? This is **label assignment**, and modern YOLO uses a smart, learnable scheme called **Task-Aligned Assignment (TAL)** that picks, for each object, the cells whose predictions are already both well-localized and well-classified. We devote a section to it in the losses chapter — it is the heart of how the model is trained.

That is the whole vocabulary. Armed with it, let’s open the codebase.

Chapter 2

The system at a glance

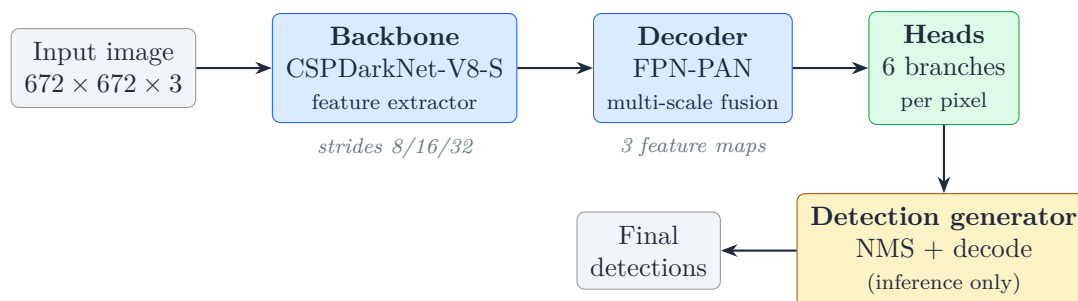
Before diving into any single file, let’s see the whole machine. This codebase takes a 672×672 RGB image and, in one forward pass, predicts three things for every object it finds:

1. a **bounding box** and a **class** (this is ordinary object detection);
2. a **polygon outline** of the object (segmentation);
3. the **distance** to the object in meters.

It is a faithful TensorFlow 2.16 re-implementation of Ultralytics YOLOv8, extended with two extra prediction “heads”. The assembly happens in `models/yolo_v8.py` (`build_yolov8`).

2.1 The end-to-end path

Every image flows through four stages:



> Beginner’s note

A **backbone** is a stack of convolutional layers that turns raw pixels into compact “feature maps” — grids of numbers where each cell summarizes a patch of the image. A **decoder** (here a feature pyramid) mixes information across resolutions so the model can see both small and large objects. The **heads** are the final thin layers that read those features and output the actual predictions. We unpack each of these in the architecture chapter.

2.2 Three tiers, one codebase

The same code supports three configurations, chosen by an experiment YAML file. They differ only in which heads are turned on:

Config (<code>configs/experiments/yolo/</code>)	Heads	Task
<code>yolov8_bbox.yaml</code>	box + cls	detection only
<code>yolov8_poly.yaml</code>	box + cls + polygon	detection + segmentation
<code>yolov8_poly_dist.yaml</code>	all 6 heads	detection + segmentation + distance

The full tier, `yolov8_poly_dist.yaml`, is the authoritative reference configuration — when in doubt about a hyperparameter, that file is the source of truth.

2.3 The six heads

Every head produces a prediction at *every pixel* of *every* one of the three feature-map scales. Here is what each one outputs.

Head	Channels	Meaning
box	64	box position, as a “distribution” — 4 sides $\times$ 16 bins (DFL)
cls	39	one logit per class
poly_angle	24	per-vertex sub-bin angle offset (one per polygon bin)
poly_dist	24	per-vertex radial distance (one per polygon bin)
poly_conf	24	per-vertex “is there a vertex here?” gate
dist	1	object distance, on a log scale

Under the hood

The heads live in `models/head.py` (`YoloV8Head`). The three polygon heads (`poly_angle`, `poly_dist`, `poly_conf`) together encode a 24-vertex outline in the “radial” PolyYOLO format — the subject of its own chapter. The box head’s 64 channels are 4×16 : each of the four box sides is predicted as a little probability distribution over 16 distance bins, a trick called *Distribution Focal Loss*. The heads are pinned to `float32` even when the rest of the model runs in `mixed_bfloat16`.

2.4 What “training” produces

During training the model runs in a mode where the heads emit *raw* numbers and a loss compares them to ground-truth labels. During inference (`deploy=True`) a final stage — the **detection generator** (`models/detection_generator.py`) — decodes those raw numbers into clean boxes, runs **non-maximum suppression** to remove duplicates, and converts the polygon and distance outputs into human-readable form.

> Beginner’s note

Non-maximum suppression (NMS) is a clean-up step. A detector typically fires on the same object from several neighboring pixels, producing a cluster of overlapping boxes. NMS keeps the highest-scoring box in each cluster and suppresses the rest, so each object is reported once.

With the map in hand, the next chapter builds up the detection ideas themselves — so that when we open the code, every term already means something.

Chapter 3

The data pipeline

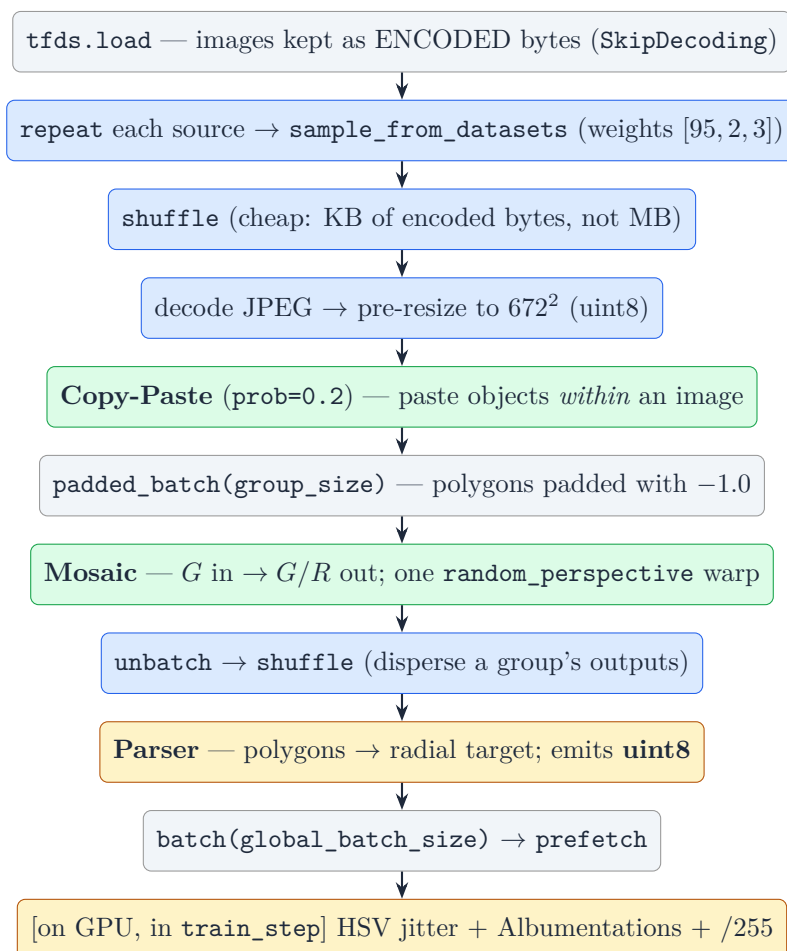
A model is only as good as the data it sees. This codebase has an unusually rich input pipeline: it samples from several datasets, augments aggressively, and even merges a *second* dataset (for distance) into each training batch. Everything lives under `data_pipeline/` and is assembled in `input_reader.py`.

> Beginner's note

In TensorFlow, data loading is built as a `tf.data pipeline`: a chain of operations (read → decode → shuffle → augment → batch) that streams examples to the GPU. Each stage runs in parallel on CPU worker threads while the GPU trains on the previous batch, so data prep and training overlap.

3.1 The stage order

Here is the whole detection pipeline, top to bottom. The order is not arbitrary — each stage depends on the previous one.



Under the hood

Two design choices keep this fast on a CPU-limited host. (1) **SkipDecoding**: images travel through the shuffle buffer as compressed JPEG bytes (kilobytes), and are only decoded inside the parallel `map` — a 16k-element shuffle buffer stays cheap instead of holding gigabytes of decoded pixels. (2) **Color augmentation is moved to the GPU**: the parsers emit `uint8` images and the HSV/Albumentations//255 work happens once per batch inside `train_step` ([data_pipeline/batch_color_aug.py](#)), cutting host→device traffic 4×.

3.2 Sampling from several datasets

The detection stream is not one dataset but several (`cleaner_polygon2026`, `field_misrecog2026`, `station_misrecog`), blended with fixed **sampling weights**. Each source is `repeat()`-ed first so the blend is a stationary, infinite stream — the weights stay constant forever rather than drifting as a finite source runs out.

3.3 Copy-Paste, then Mosaic

Two augmentations stitch content together, and *order matters*.

Copy-Paste ([copy_paste.py](#)) pastes a cut-out object (from a separate RGBA dataset, the alpha channel being the mask) onto the current image. It runs *first*, because it augments *within* a single image. **Mosaic** then stitches four whole images together, so it must come after.

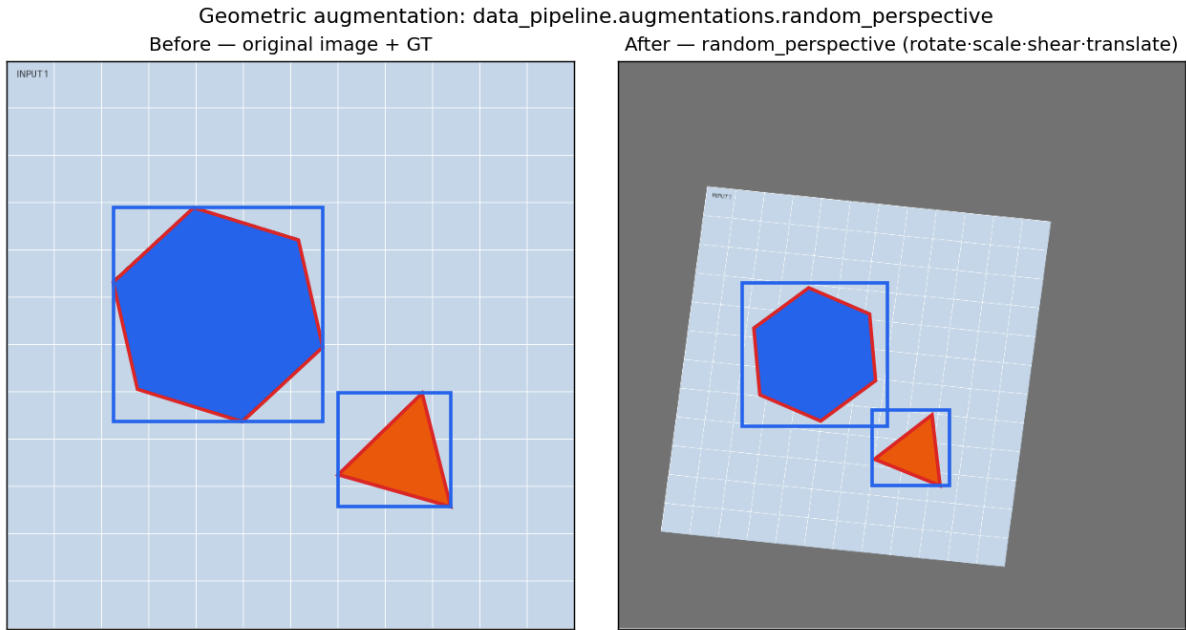


Figure 3.1: The one geometric transform, `random_perspective` (rotation, scale, shear, translate), rendered by the actual code in `data_pipeline/augmentations.py`. The box (blue) and polygon (red) ground truth are transformed by the same matrix and clipped to the image edge. This same warp is applied to single images and to mosaics.

3.3.1 Mosaic in depth

Mosaic builds a $2\times$ -size canvas, places four images toward a random center, and then applies *one* `random_perspective` warp that crops it back to 672×672 .

Why this codebase does it this way

The mosaic assembles a canvas and warps *once*, rather than warping each of the four source images separately and compositing. The two are geometrically identical, but the “warp-once” canvas form was measured roughly $3\times$ faster on the production CPU (the warp op is several times more expensive per pixel than a plain resize, and the alternative pays four full warps per mosaic). On a CPU-bound pipeline this directly sets the epoch time.

The group / diversity mechanism

Mosaic is implemented as a map over a `padded_batch(group_size)` group. A group of G decoded images emits G/R output samples, where $R = \text{decodes_per_output}$.



At the default $R = 4$ (“stock YOLO”), the width-4 windows *tile* the permutation: every output uses four distinct images with zero reuse. Lowering R makes the windows overlap, so each image is reused in $4/R$ outputs with varied partners — less decode work, slightly less diversity. Crucially, the number of *training samples* per epoch is unchanged; R only changes how many source images are decoded.

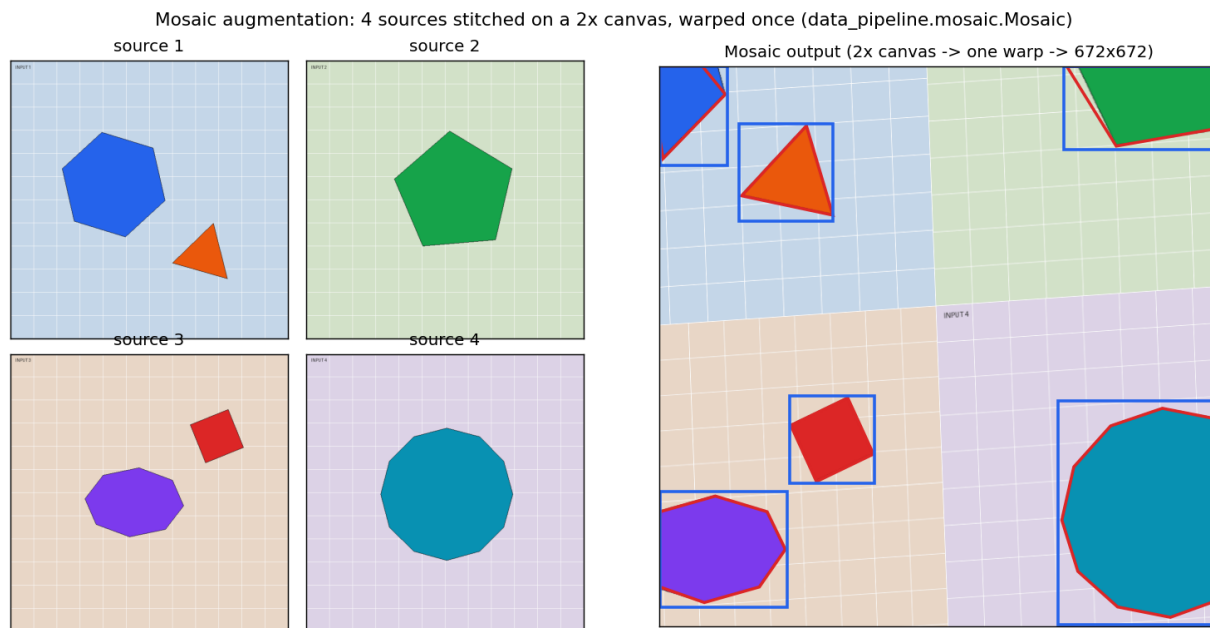


Figure 3.2: The real `Mosaic._mosaic` output. Four source images are resized and placed into the four quadrants of a 1344×1344 canvas, then a single warp produces the final image. Ground-truth boxes and polygons are carried through the *same* matrix, so labels stay perfectly aligned.

Gotcha

The `padded_batch` step pads polygon rows with -1.0 , *not* the default 0.0 . Why? 0.0 is a perfectly valid top-left vertex coordinate — zero-padded rows would be read as real vertices and corrupt the polygon target. The value -1.0 is the reserved “no vertex here” sentinel throughout the codebase.

3.4 Flip and color

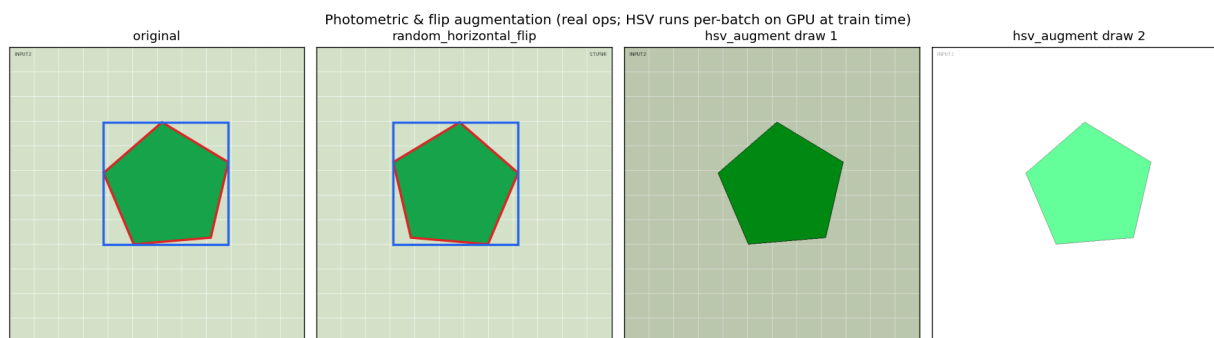
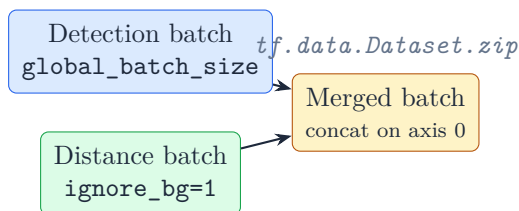


Figure 3.3: Horizontal flip (real `random_horizontal_flip` — note the box and polygon x-coordinates flip too) and two draws of HSV jitter (real `hsv_augment`). The brightness term here is *additive*, a deliberate match to the original codebase. HSV and Albumentations actually run per-batch on the GPU at train time; this figure calls the same functions directly to show what they do.

3.5 Merging the distance dataset

Here is the cleverest part of the pipeline. Distance labels come from a *different* dataset (`servingbot_polygon`) than detection. Rather than build a separate training job, the two streams are zipped and concatenated on the **batch dimension**:



The merged batch flows through the same model. Distance-only samples carry `ignore_bg=1`, which tells the loss to skip class supervision on them (they have no detection labels). This stream is **training only** — there is no distance validation path, so the distance head is trained but not scored during validation.

Under the hood

Three shuffle stages use *distinct* seeds (`seed`, `seed+1`, `seed+2`): the detection source shuffle, the copy-paste source shuffle, and the post-unbatch shuffle. Distinct seeds keep the stages decorrelated — a shared seed would pair the same images every epoch and partially undo each shuffle’s job.

Chapter 4

Polygons: the PolyYOLO radial representation

The polygon head is the most distinctive part of this codebase, so it gets its own chapter. The goal: predict an object's outline as 24 points, cheaply enough to be just three small per-pixel heads.

4.1 The radial idea

Instead of storing 24 arbitrary (x, y) points, PolyYOLO fixes the *angles* and predicts only the *radii*. Imagine standing at the object's box center and shooting 24 rays outward, one every 15° ($360^\circ/24$). For each ray, predict how far the boundary is. That distance is the only number you need per vertex — the angle is implied by the bin index.

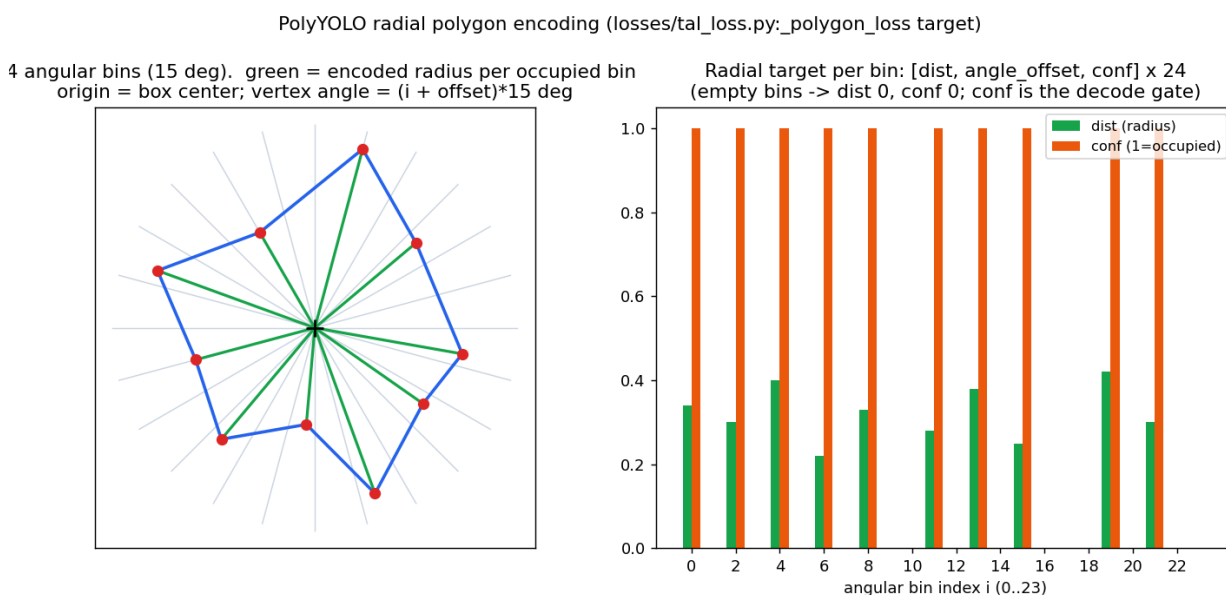


Figure 4.1: The radial encoding. Left: 24 angular bins (15° each) around the box center; the green segment in each occupied bin is the encoded radius to the polygon boundary (blue). Right: the per-bin target — a radius (*dist*) and a validity flag (*conf*) for each of the 24 bins. This is exactly the target built for the loss.

Under the hood

Each vertex is actually *three* numbers, so the target is $[24 \times 3] = 72$ values per object, interleaved as `[dist, angle, conf]`:

- `dist` — the radius (how far the boundary is along this ray);
- `angle` — a **sub-bin offset** in $[0, 1)$: the exact vertex angle is $(i + \text{offset}) \cdot 15^\circ$, so the vertex isn't snapped to the bin center;
- `conf` — 1 if this bin holds a vertex, 0 if empty.

This is built in `losses/tal_loss.py:_polygon_loss`. The three model heads `poly_dist`, `poly_angle`, `poly_conf` predict these channels.

> Beginner's note

Why a sub-bin offset instead of just the bin index? With 15° bins, snapping every vertex to its bin center would quantize the outline coarsely. The offset recovers the *exact* angle within the bin, so the 24 vertices can sit anywhere on the boundary — much smoother outlines for the same 24 bins.

4.2 Why confidence sees every bin

Not every object has a vertex in every direction (a thin object leaves many bins empty). The `conf` channel is the gate that decides, at decode time, which bins emit a vertex (threshold 0.4).

Why this codebase does it this way

The `conf` loss is averaged over **all 24 bins** (occupied $\rightarrow 1$, empty $\rightarrow 0$), while `dist` and `angle` are averaged over occupied bins only. The reason is subtle and was found the hard way: `conf` is the gate, so it must be *trained on negatives* (empty bins) to learn to output low confidence there. An earlier version masked `conf` to occupied bins only — so empty bins never got a “you should be off” signal, their confidence drifted above the 0.4 threshold, and the decoder emitted spurious vertices: the “spiky star” artifacts seen in validation overlays. `dist/angle` stay masked because a radius/angle on an empty bin is undefined — there is no vertex to regress toward.

4.3 Arc-length resampling: making bins track real shapes

There is a catch. Datasets often store polygons with few vertices — a rectangle might be just its 4 corners. Drop 4 corners into 24 angular bins and only ~ 4 bins get a vertex; the radial target becomes a diamond, and the long edges (which cross many empty bins) are never learned.

Under the hood

`resample_polygons` (in `data_pipeline/augmentations.py`) walks the closed contour and places `K=64` points at equal arc-length intervals, interpolating *on the edges* — it does not just sub-sample stored vertices. This runs at decode time (`parser.resample_points: 64`), so every downstream stage carries a compact `[N, 128]` polygon instead of the raw stored width (which can be thousands of columns).

4.4 Three formats across the stack

The same polygon wears three different “costumes” as it moves through the code. Keeping them straight is essential.

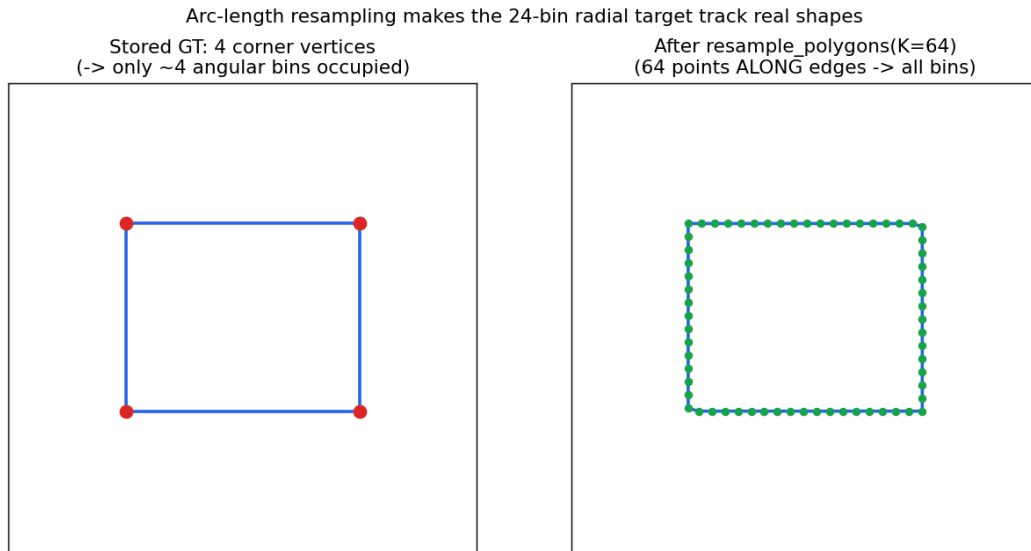
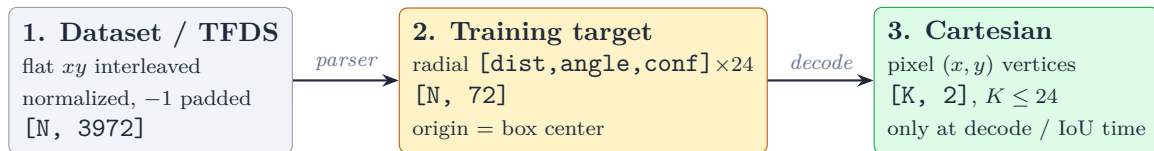


Figure 4.2: The fix, run through the real `resample_polygons`. A 4-corner rectangle (left) is resampled to 64 points spaced uniformly *along* the contour (right). The new points lie on the edges, so every angular bin the boundary crosses now gets a vertex — the rectangle encodes as a rectangle, not a diamond.



Stage	Shape	Where
TFDS input	[N, 3972] flat xy , -1 pad	dataset decoders
Training target	[N, 72] radial	loss / parser
Prediction output	[B, max_det, 24, 3] activated	detection generator
Cartesian (transient)	[K, 2] pixels	only at IoU/eval/viz

Gotcha

In the model's *prediction* output, the polygon `conf` values are already sigmoid-activated by the detection generator — they are **not** raw logits. Apply your 0.4 threshold directly. (And remember validity is keyed off the -1.0 sentinel, not " ≥ 0 ": a vertex can legitimately be slightly negative after a mosaic overflow and is still real.)

4.5 Decoding a predicted polygon

At inference, for each bin i whose `confi ≥ 0.4`:

$$\text{angle} = (i + \text{offset}_i) \cdot 15^\circ, \quad \text{radius} = \text{dist}_i, \quad (x, y) = \text{box center} + \text{radius} \cdot (\cos, \sin) \text{ angle}.$$

Bins below threshold are dropped. The result is up to 24 Cartesian vertices, used for visualization and for the polygon IoU metric.

> Beginner’s note

Known limits of the format. Because the origin is the box center and each bin keeps only the *farthest* crossing, the radial format cannot represent holes, interior concavities, or shapes whose center lies outside the polygon. These are properties of the representation itself, not bugs — it trades exactness for a compact, easy-to-predict encoding.

Chapter 5

The model architecture

Now we open the network itself. It has three parts — backbone, decoder, heads — plus a post-processing stage for inference. Everything is assembled in `models/yolo_v8.py:build_yolov8`.

5.1 Backbone: turning pixels into features

The backbone is **CSPDarkNet-V8** (`models/backbone.py`), built from **C2f** blocks and an **SPPF** block. It is the “small” (–s) size, set by `depth_scale=0.33` and `width_scale=0.5`. It emits three feature maps, at levels “3”, “4”, “5” — strides 8, 16, 32.

> Beginner’s note

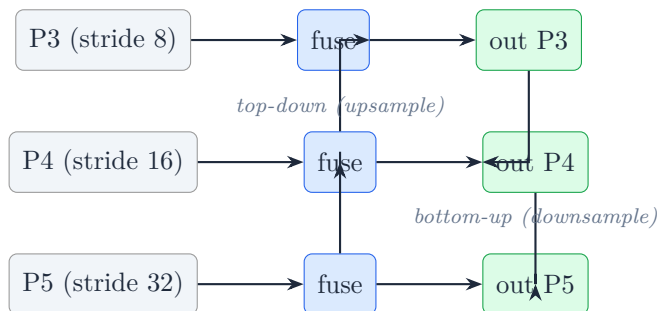
A **C2f block** is YOLOv8’s main building block: it splits the channels, runs several small bottleneck convolutions, and concatenates everything back. This reuses features cheaply (a “cross-stage partial” design). **SPPF** (Spatial Pyramid Pooling – Fast) pools the features at several window sizes and stacks them, giving the network a large receptive field without huge kernels.

Gotcha

Even though the shipped YAML sets `depth_scale: 1.0` and `width_scale: 1.0`, the model is still the *small* one: `model_id: cspdarknetv8s` takes precedence over those scale fields. Don’t be fooled by the 1.0s.

5.2 Decoder: the FPN-PAN

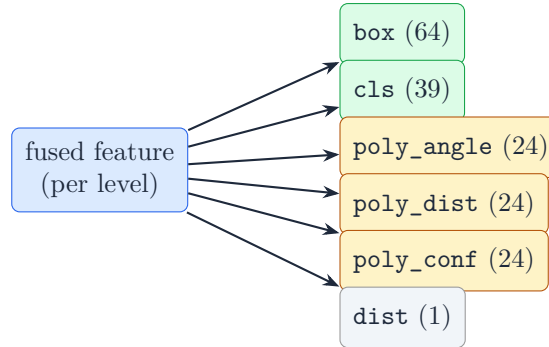
The three backbone maps go into an **FPN-PAN** decoder (`models/decoder.py`), which fuses information across scales in two passes.



The top-down path carries strong semantic signal from coarse levels down to fine ones; the bottom-up path carries precise localization back up. Each fusion uses a **C2f** stack; activations are ReLU throughout. The output is one fused feature map per level, fed to the heads.

5.3 The six heads

At every pixel of every level, six small branches predict (`models/head.py`, `YoloV8Head`):



Under the hood

Smart bias initialization. The class head's bias is initialized to $\log(5/\text{num_classes}/(\text{input_size}/\text{stride})^2)$ so that, before any training, the model predicts a low, sensible object probability per cell (this stabilizes the first steps). The box bias is set to 1.0. When warm-starting, only the backbone and decoder weights load — the heads start fresh.

5.4 How the box head works: DFL

The box head does not regress four numbers directly. Each of the four sides (left, top, right, bottom) is predicted as a **probability distribution over 16 distance bins**, and the decoded distance is the *expected value* of that distribution. This is **Distribution Focal Loss (DFL)**.

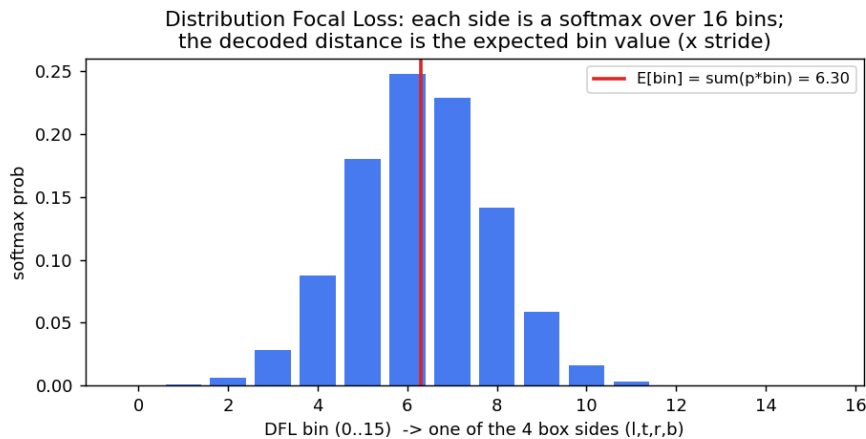


Figure 5.1: DFL: one box side as a softmax over 16 bins. The predicted distance is $\sum_i p_i \cdot i$ (the expected bin), then multiplied by the level's stride to get pixels. Predicting a distribution rather than a point lets the model express uncertainty and yields sharper, better-calibrated box edges.

So the box head's 64 channels are $4 \text{ sides} \times 16 \text{ bins}$. The decode (softmax $\rightarrow$ expected value) is a fixed 1×1 convolution in `models/detection_generator.py`: `_decode_dfl`.

5.5 Anchors and strides

The model is **anchor-free**: one anchor point per cell, at the cell center $(i + 0.5) \cdot \text{stride}$. The box head predicts the distances from that point to the four sides; `dist2bbox` turns those into corner

coordinates. Anchor points are built on the fly inside the loss and the detection generator — they are not learned.

5.6 Inference: the detection generator

During training the heads emit raw numbers. For inference (`deploy=True`) the **detection generator** decodes everything into final results:

1. DFL decode $\rightarrow$ `xyxy` boxes (times stride);
2. per-class greedy **NMS** (`max_boxes=300`, `nms=0.65`, `score=0.05`);
3. polygon decode: (`conf`, `dist`, `angle`) sigmoid/softmax-activated, vertices reconstructed for bins above threshold;
4. distance decode: exp from log-scale, clipped to `[0.5, 10.0]` meters.

> Beginner's note

Why log-scale distance? Predicting $\log(\text{distance})$ instead of raw meters makes the model's error *relative* — being off by 0.5 m matters more for a 1 m object than a 9 m one. The log transform builds that intuition into the loss, and the output is exponentiated back to meters at decode time.

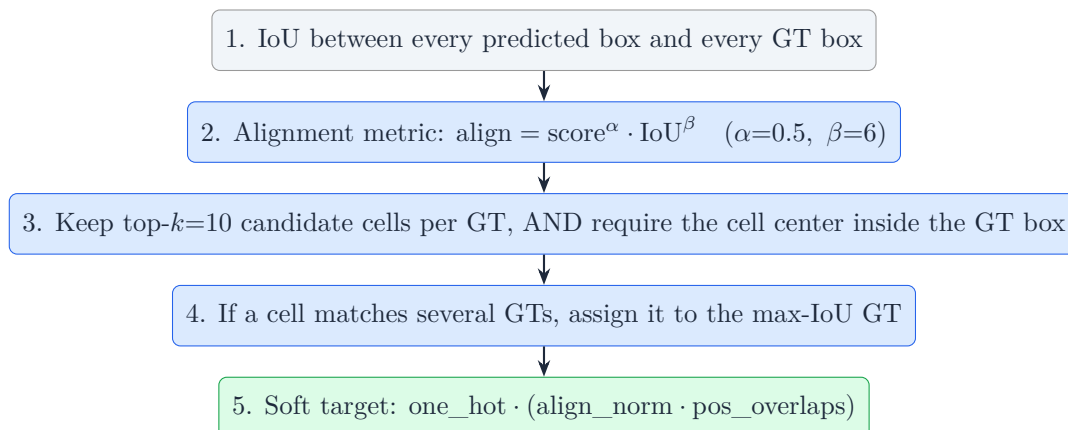
Chapter 6

The losses

This chapter is the heart of training. The total loss combines a task-aligned detection loss with polygon and distance terms. It lives in `losses/`, entry point `tal_loss.py:TaskAlignedLossExtended`.

6.1 First, the assignment problem

The model predicts at thousands of grid cells; an image has a handful of objects. Before we can compute any loss, we must decide *which cells are responsible for which object*. A bad rule wrecks training. YOLOv8 uses **Task-Aligned Assignment (TAL)** (`losses/tal_assigner.py`), which picks the cells whose predictions are already both well-localized *and* well-classified.



> Beginner's note

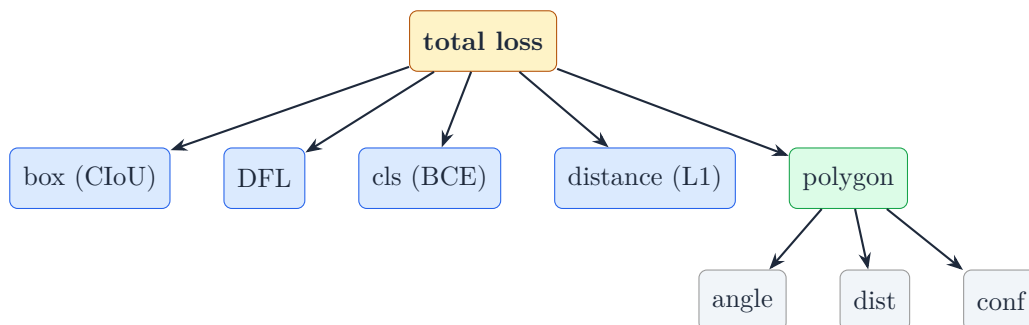
“Task-aligned” means the assignment rewards cells that do *both* jobs well at once. A cell that draws a tight box but guesses the wrong class — or names the right class but in the wrong place — scores low on $\text{score}^\alpha \cdot \text{IoU}^\beta$ and is not chosen. This couples the classification and localization tasks so they improve together.

Under the hood

The assignment is computed under `stop_gradient` — it selects targets but no gradient flows through the selection. The classification target is *soft*: instead of a hard 1, the positive class gets $\text{align_norm} \times \text{pos_overlaps}$, where `pos_overlaps` is the per-GT max IoU. This scales the class target by localization quality, exactly matching the Ultralytics recipe.

6.2 The loss tree

The total loss is a weighted sum of five families of terms:



6.2.1 Box: CIoU and DFL

CIoU is IoU plus penalties for center distance and aspect-ratio mismatch — $1 - (\text{IoU} - \rho^2/c^2 - \alpha v)$ — a smoother, better-behaved box loss than plain IoU. **DFL** trains the 16-bin distributions of the box head toward the true side distances. Both are weighted per-anchor by how aligned each anchor is, so well-matched cells dominate the box gradient.

6.2.2 Classification

Binary cross-entropy with logits, summed over the 39 classes. On distance-only samples (`ignore_bg=1`) the class loss is masked to foreground anchors, since those samples carry no detection labels.

6.2.3 Polygon: three sub-losses

The polygon terms (`losses/polygon_loss.py`) train the radial target from Chapter 4:

- **angle** — BCE on `sigmoid(pred)` toward the sub-bin offset, averaged over *valid vertices*;
- **dist** — squared error $(\text{target} - \text{softplus}(\text{pred}))^2$, averaged over *valid vertices*;
- **conf** — BCE on per-bin validity, averaged over *all 24 bins*.

6.2.4 Distance

L1 error on log-scale distance, masked to entries with a valid distance label (sentinel `-10.0` marks “no distance”), normalized by the total GT object count.

6.3 Gains and normalization

Each term is scaled by a **gain** from the YAML. But the terms do not share the same denominator, so the gains are *not* directly comparable across heads — a crucial subtlety.

Term	Gain	Normalizer / reduction
box CIoU / DFL	7.5 / 1.5	<code>target_scores_sum</code> , per-anchor weighted
cls	0.5	<code>target_scores_sum</code>
distance	1.0	<code>num_objs</code> (total batch GT count)
poly angle	0.4	<code>num_objs</code> , mean over valid vertices
poly dist	0.45	<code>num_objs</code> , mean over valid vertices
poly conf	0.2	<code>num_objs</code> , mean over all 24 bins

An overall `poly_gain=0.5` multiplies the summed polygon loss. The assignment parameters are `tal_alpha=0.5`, `tal_beta=6.0`, `topk=10`.

Gotcha

Because the denominators differ (`target_scores_sum` for detection vs `num_objs` for polygon/distance), you cannot reason about relative importance by comparing gains alone. If you change a masking rule or a reduction, the magnitude of that term shifts and its gain must be re-tuned. These conventions are pinned by `tests/test_polygon_loss_conventions.py` precisely so they don't drift silently.

Under the hood

The `loss` returns a 9-tuple: `(total, box, dfl, cls, dist, poly, poly_angle_raw, poly_dist_raw, poly_conf_raw)`. The last three are the un-gained polygon sub-losses, logged separately to TensorBoard (`train/poly_angle_loss`, etc.) so you can see *which* polygon component is misbehaving — they do not re-enter `total`.

Chapter 7

Training: optimizer, schedules, and the loop

We have a model and a loss. This chapter is how the codebase actually *trains* — the optimizer, the learning-rate schedule, the weight-averaging trick, and the custom training loop. The entry point is [scripts/run_train.py](#); the loop lives in [train/trainer.py](#).

7.1 The optimizer: SGD with warmup

Training uses **SGD with Nesterov momentum** ([optimizers/sgd_warmup.py](#)). Two features make it specific to this codebase.

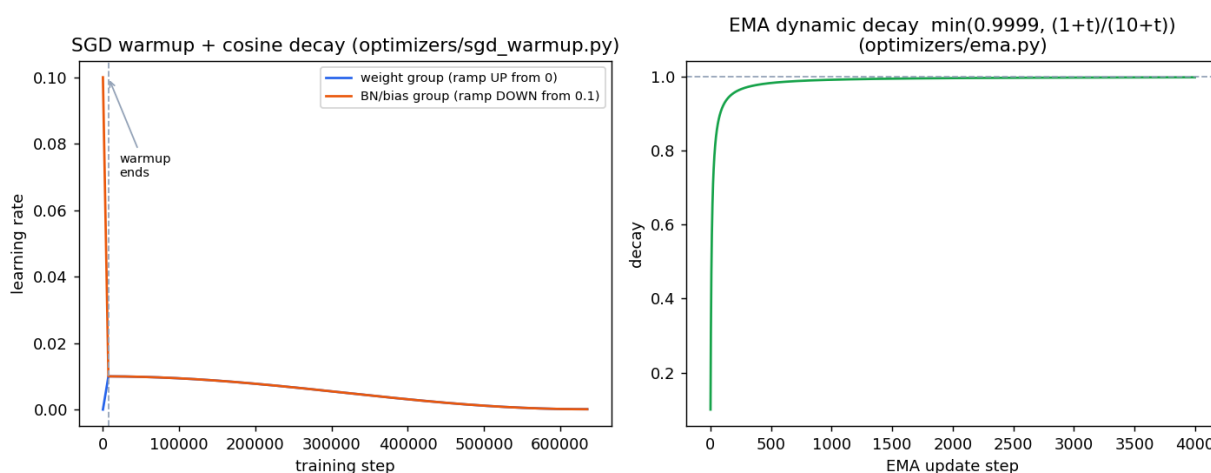


Figure 7.1: Left: the learning-rate schedule, computed from the actual optimizer formulas. The weight group ramps *up* from 0 during warmup, then follows a cosine decay. The BN/bias groups ramp *down* from 0.1. Right: the EMA decay, $\min(0.9999, (1+t)/(10+t))$ — it starts low (fast averaging) and rises toward 0.9999 (slow, stable averaging) as training proceeds.

> Beginner's note

Warmup means starting with a tiny learning rate and increasing it over the first few thousand steps. Fresh weights produce wild gradients; a gentle start avoids blowing up before the model settles. **Cosine decay** then smoothly lowers the rate toward the end so the model fine-tunes into a good minimum.

Why this codebase does it this way

This optimizer has **three parameter groups** — BatchNorm, biases, and weights — with different warmup behavior. Unusually, the BN/bias groups *ramp down* from an absolute LR of 0.1 (ten times the base) toward the base rate, while weights ramp up from 0. Stock Ultralytics warms biases *up* from a small value. This ramp-*down* direction matches the original codebase the model was migrated from, and the existing checkpoints were trained under it — changing it would mean retraining.

7.2 EMA: a smoothed copy of the weights

The trainer keeps an **Exponential Moving Average** of the weights (`optimizers/ema.py`). After each step, a shadow copy is nudged toward the current weights with a decay that grows over time. The EMA weights are usually smoother and generalize better, so they are **swapped in for evaluation** and swapped back out afterward.

Under the hood

The decay is dynamic: $\min(0.9999, (1 + t)/(10 + t))$, incremented before it is read (matching Ultralytics' `ModelEMA`). Evaluation calls `swap_in` before and `swap_out` after, so reported metrics reflect the averaged weights while training continues on the raw ones. Warm-starting a new run also loads from the EMA shadows — they hold the complete weights, including list-tracked C2f sublayers the plain object graph can miss.

7.3 The training loop and epoch accounting

The loop is custom (not Orbit) so it can do several project-specific things: swap EMA weights around validation, merge the detection+distance streams, save checkpoints at epoch boundaries, handle preemption (SIGTERM), and auto-resume from the newest checkpoint.

Under the hood

The training stream is **infinite**, so an “epoch” is defined by the trainer, not by data exhaustion. It runs exactly `steps_per_loop` steps per epoch, where `steps_per_loop = train_total_examples // batch_size`, from one persistent iterator. Everything derived is then consistent by construction: `decay_steps = steps_per_loop × epochs` and `checkpoint_interval = steps_per_loop` (one epoch). After a mid-epoch resume, only the remainder to the next epoch boundary is run, so boundaries stay at exact multiples.

Gotcha

`decay_steps` is an explicit field in the YAML, not derived. If it disagrees with `steps_per_loop × epochs`, the cosine schedule will not finish where the training does. `run_train.py` *warns* at startup if they diverge — watch for that line.

7.4 Mixed precision and XLA

The reference tier runs in `mixed_bfloat16`: most of the network computes in 16-bit `bfloat16` (faster, less memory), while the prediction heads and the loss are pinned to `float32` for numerical safety — so no loss scaling is needed. **XLA** (a graph compiler) is an optional A/B toggle. Both are applied in `run_train.py: _apply_runtime_config` before any op runs.

> Beginner's note

bfloat16 is a 16-bit float with the same exponent range as 32-bit but fewer mantissa bits. It halves memory and speeds up matmuls on modern GPUs, while its wide range avoids the overflow problems that plague the older `float16`. Keeping the loss in `float32` preserves accuracy where it matters most.

7.5 Multiple GPUs

Data-parallel training with `MirroredStrategy` is supported and **numerically identical** to single-device. The reference config defaults to `one_device` on one GPU (avoiding `MirroredStrategy`'s overhead). The correctness hinges on two things: the loss normalizers (`num_objs`, `target_scores_sum`) are all-reduced to *global* counts, and gradients are all-reduced across replicas — both no-ops on a single GPU, so single-GPU runs are byte-for-byte unchanged.

7.6 What you can watch while it trains

TensorBoard gets rich logging: every scalar carries a markdown description (name + formula), per-class detection metrics are tagged by class name, the three polygon sub-losses are logged separately, and `train/data_wait_ms` vs `train/throughput_img_per_s` let you tell whether the bottleneck is the data pipeline or the GPU. Augmented training images are logged each epoch under `train/augmentations` (geometry + labels, before the GPU color pass).

Chapter 8

Configuration: how a run is described

Everything about a run — model size, augmentation, gains, schedule — lives in one **experiment YAML**. There is no hidden state; the YAML is the authoritative hyperparameter source. This chapter explains how it loads and what the main knobs are.

8.1 YAML to dataclasses

`configs/yaml_loader.py:load_config` reads the YAML and maps it onto plain dataclasses in `configs/model_config.py`. The mapper is **hand-rolled** (not `dacite`, despite the dependency being listed).

Gotcha

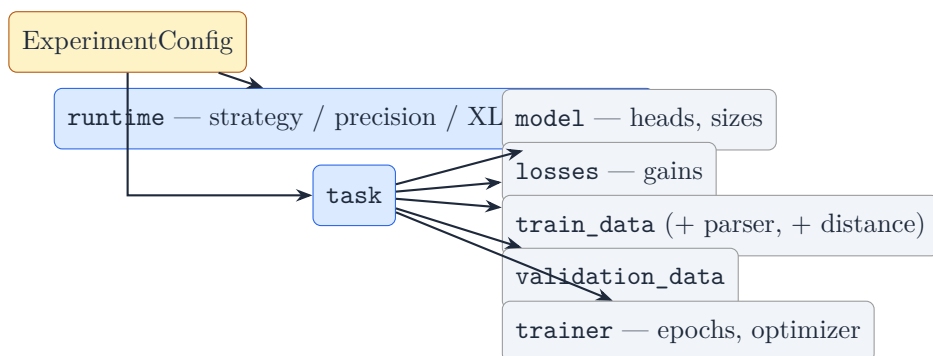
Unknown keys outside the `runtime/losses` sections are **silently ignored**. A typo in a field name will not raise an error — it just won't take effect. If a setting seems to do nothing, check the spelling against the dataclass field names.

8.1.1 Inheritance

A config can set `base: <path>` to inherit another config and deep-merge its own keys on top (override wins, dicts merge). For example, the `bf16/XLA` variant is just a thin overlay:

```
base: yolov8_poly_dist.yaml
runtime:
  enable_xla: true
```

8.2 The structure



8.3 The knobs you will touch most

Field	Default	What it does
<code>model.input_size</code>	[672,672,3]	live model input; smart-bias init assumes it
<code>model.num_classes</code>	39	head width + checkpoint-migration rule
<code>model.angle_step</code>	15	polygon bin size; <code>output_poly_size == 360/angle_step</code>
<code>train_data.global_batch_size</code>	128	detection batch (distance batch merged on top)
<code>parser.resample_points</code>	64	arc-length polygon resample target
<code>mosaic.group_size</code>	32	mosaic source pool per group
<code>mosaic.decodes_per_output</code>	4	R: 4 = stock YOLO; lower = more reuse, less decode
<code>trainer.train_epochs</code>	300	one epoch = one pass of training samples
<code>losses.iou_gain / cls_gain / dfl_gain</code>	7.5/0.5/1.5	detection gains

8.4 Derived fields — don't hand-edit

`steps_per_loop`, `train_steps`, `validation_steps`, and `checkpoint_interval` are *computed* from `train_total_examples` and the batch sizes. The resolved values print in the startup banner. Editing them by hand only desynchronizes the schedule.

8.5 Validated invariants

Before training starts, `run_train.py:_validate_config` refuses to run if any of these fail — a fast, friendly guard against silent misconfiguration:

- `init_checkpoint` (if set) exists;
- `output_poly_size == 360 // angle_step`;
- `group_size >= 4` and `group_size % decodes_per_output == 0`;
- `output_dir` is creatable; each referenced TFDS data dir exists.

It additionally *warns* (not fatal) if `decay_steps != train_steps`.

> Beginner's note

Why dataclasses + YAML? The YAML is what a human edits and version-controls; the dataclasses give the code type-checked, autocomplete-friendly access with sane defaults. The loader is the bridge. Keeping all hyperparameters in one declarative file makes runs reproducible: the config *is* the experiment.

Chapter 9

Evaluation: measuring quality

How do we know the model is any good? The codebase computes three families of metrics — detection, polygon, and distance — in `eval/`. This chapter is a plain-language glossary, so the numbers in TensorBoard and `tools/eval` mean something.

9.1 Detection metrics (COCO)

These follow the standard COCO protocol (`eval/coco_metrics.py`).

Metric	Meaning
mAP	mean Average Precision averaged over IoU thresholds 0.50:0.95 (the headline COCO number)
mAP50	AP at a single, looser IoU threshold of 0.5
AR100	average recall with up to 100 detections per image
F1score50	macro-averaged peak F1 at IoU 0.5 — the best-checkpoint selection metric
precision50 / recall50	precision / recall at each class’s peak-F1 operating point

> Beginner’s note

Precision, recall, F1. Precision = “of the boxes I predicted, how many were right?” Recall = “of the real objects, how many did I find?” They trade off as you move the confidence threshold. **F1** is their harmonic mean; the “peak F1” is the best balance over the whole curve. **Average Precision** summarizes the entire precision–recall curve as one number; **mAP** averages it over classes and IoU thresholds.

Why this codebase does it this way

The best checkpoint is chosen by **F1score50**, not **mAP**. F1 at a single operating point reflects deployable detection quality (a threshold you would actually ship), and it is more stable run-to-run for this multi-task model than the stricter mAP average. The per-category table (`--per_category`) exposes which few classes drag the macro average down.

9.2 Polygon metrics

Predictions are matched to ground truth by **box** IoU > 0.5, then the polygon **mask** IoU is computed by rasterizing both outlines (`cv2.fillPoly`).

<code>poly_mIoU</code>	mean polygon mask IoU over matched pairs — pure segmentation quality
<code>poly_recall50</code>	fraction of GT objects matched at box IoU 0.5

The polygon conf gate (0.4) is a single shared constant (`eval/polygon_metrics.DEFAULT_POLY_CONF_THRESH`) used by both the metric and the visualization overlays, so they can never disagree.

9.3 Distance metrics

Each GT is matched to its highest-IoU detection (≥ 0.5); errors are in meters (predictions are exp'd from log-space first), split near/far at 5 m.

<code>dist_mae</code>	mean absolute error, meters
<code>dist_rmse</code>	root-mean-squared error (penalizes large misses)
<code>dist_absrel</code>	mean relative error $ \text{pred} - \text{gt} /\text{gt}$
<code>dist*_near</code> / <code>*_far</code>	the same, restricted to < 5 m / ≥ 5 m

Gotcha

Distance is only scored when the eval split carries distance labels. The shipped distance dataset is **training-only**, so during normal validation the distance head is *not* evaluated — a distance regression would not show up in val metrics. The evaluator code exists; it simply isn't wired into the default val loop.

9.4 Running it

```
python -m tools.eval \
    --config configs/experiments/yolo/yolov8_poly_dist.yaml \
    --checkpoint /run/ckpt-100000 --split val --per_category
```

Evaluation automatically uses the EMA weights. `tools/continuous_eval.py` can watch a run directory and evaluate each new checkpoint automatically, appending to `eval_log.jsonl`.

Chapter 10

Tools, testing, and on-device export

A research codebase is more than the model. This chapter surveys the supporting cast: the runnable scripts, the test suite, checkpoint migration, and the export path that puts the model on an embedded device.

10.1 The scripts you run

All under `tools/` and `scripts/`; full details and copy-paste commands are in `docs/scripts.md`.

Command	What it does
<code>scripts.run_train</code>	launch training (prefer <code>tools/train_supervisor.sh</code> for long runs)
<code>tools.eval</code>	COCO / polygon / distance metrics on a checkpoint
<code>tools.infer</code>	draw box + polygon overlays on arbitrary images
<code>tools.continuous_eval</code>	auto-evaluate new checkpoints in a run directory
<code>tools.export_saved_model</code>	host/server SavedModel (NMS baked in, $[0, 1]$ input)
<code>tools.device.export_device_dlc</code>	on-device SNPE/DLC export
<code>tools.checkpoint_migration</code>	migrate / warm-start checkpoints
<code>tools.benchmark_pipeline</code>	data-pipeline throughput
<code>tools.pipeline.diagnose_pipeline</code>	stage-by-stage pipeline timing

10.2 Checkpoint migration and warm-starting

`tools/checkpoint_migration.py` loads weights into the model from an existing checkpoint, auto-detecting the kind:

Source	Strategy	What transfers
legacy TF2-Vision ckpt	frozen	backbone + decoder (+ head if classes match)
this codebase's own ckpt	native	requested modules, from EMA weights

Gotcha

Warm-start from a checkpoint that carries EMA (a periodic `ckpt-N` or a `best_*`), **not** a bare model export. The plain model object graph omits some list-tracked C2f block variables (a Keras quirk), so only the EMA shadows hold the *complete* weights. The `native` strategy loads via that EMA path for exactly this reason.

10.3 Testing

The `pytest` suite (`tests/`) runs in **eager mode** and is split into `unit/` (components — backbone, decoders, EMA, the assigner, the evaluators), `integration/` (full pipeline, checkpoint migration, a real 2-replica multi-GPU test on virtual CPU devices), `smoke/` (a 10-step training loop), and top-level component tests (losses, mosaic, parser, polygon preprocessing).

Under the hood

Two conventions make the tests trustworthy. (1) Unit/integration tests **never** require the real datasets — they build synthetic tensors inline, so CI runs anywhere. (2) Tests are *discriminating*: they pin a value or relationship that *fails* on the buggy behavior and passes once fixed (see `tests/test_loss_reference_parity.py`), rather than merely asserting “runs without error”.

```
# fast, no datasets (what CI runs):
pytest tests/unit tests/integration -q
# the multi-GPU test needs a fresh process:
pytest tests/integration/test_multigpu.py -q
```

10.4 On-device export (Qualcomm SNPE / DLC)

The model ultimately runs on an embedded device. `tools/device/export_device_dlc.py` emits a TensorFlow SavedModel laid out as a **drop-in replacement** for a legacy on-device DLC, so the existing SNPE conversion pipeline keeps working unchanged. It differs from the host export in deliberate ways: it bakes the `/255` normalization in (the device feeds raw `[0, 255]` pixels), runs in `float32`, emits one raw tensor per head (FPN levels concatenated), and promotes each head to a cleanly named top-level graph node the SNPE extractor expects.

Gotcha

The deployed device decoder stores anchors as (y, x) and expects box channels in **y-first** order $[t, l, b, r]$, while the model (and the host code) is x-first $[l, t, r, b]$. The exporter therefore **reorders the box channels** by default (`--legacy_box_order`). Feeding x-first boxes to the y-first decoder transposes every box — the kind of bug that drops on-device accuracy from 0.68 to 0.19 while every shape still looks correct. The exporter’s `--verify` flag checks this and more.

Chapter 11

Appendix

11.1 Glossary

Anchor-free

each grid cell has one anchor point (its center); the box head predicts distances to the four sides, not offsets from preset box shapes.

Backbone

the convolutional feature extractor (here CSPDarkNet-V8-S).

bfloat16

a 16-bit float with float32's exponent range; halves memory, no loss scaling needed.

CIoU

complete IoU: IoU plus center-distance and aspect-ratio penalties; the box regression loss.

C2f

YOLOv8's cross-stage-partial block: split channels, run bottlenecks, concatenate.

DFL

Distribution Focal Loss: predict each box side as a softmax over 16 bins; decode = expected bin value.

EMA

exponential moving average of the weights; a smoothed copy used for evaluation.

FPN-PAN

feature-pyramid decoder with a top-down then bottom-up fusion path.

IoU

intersection over union; overlap measure in $[0, 1]$.

Mosaic

augmentation stitching four images into one $2\times$ canvas, then one warp.

NMS

non-maximum suppression; removes duplicate overlapping detections.

PolyYOLO radial format

24 vertices at fixed 15° angles, each storing a radius, sub-bin angle offset, and validity flag.

SkipDecoding

keeps images as encoded JPEG bytes through the shuffle buffer; decode happens later in the parallel map.

SPPF

spatial pyramid pooling (fast); enlarges the receptive field.

Stride

the down-sampling factor of a feature map (8 / 16 / 32 here).

TAL

Task-Aligned Assignment: picks responsible cells using $\text{score}^\alpha \cdot \text{IoU}^\beta$.

xyyx / xyxy

box coordinate orders (y-first vs x-first); this codebase converts between them at a known seam.

11.2 Where everything lives

Directory	Contents
data_pipeline/	decoders, copy-paste, mosaic, augmentations, parsers, the distance merge
models/	backbone, decoder, the 6 heads, detection generator, assembly
losses/	TAL assigner, the total loss, polygon and distance losses
optimizers/	SGD + warmup, EMA
eval/	COCO, polygon, and distance metrics
train/	the task, the custom trainer, viz utilities
configs/	dataclasses, the YAML loader, the composable YAML fragments
scripts/	<code>run_train.py</code> entry point
tools/	eval, export, migration, benchmarking, device export, diagnostics
tests/	unit / integration / smoke / component tests
docs/	the developer documentation this guide is built from

11.3 Design decisions worth remembering

A handful of choices differ from stock YOLOv8 on purpose (see [docs/design_register.md](#) for the full list and reasoning):

- Training *filters* crowd annotations, while COCO eval *counts* them — matching the original PolyYOLO protocol.
- HSV brightness jitter is *additive*, not multiplicative.
- The BN/bias warmup LR ramps *down* from 0.1.
- Polygon `conf` loss covers *all 24 bins*; `angle/dist` cover occupied bins only.
- Polygon vertices are *arc-length resampled* to 64 points at decode time.
- Mosaic uses the *canvas* formulation (one warp), measured fastest on the target CPU.
- -1.0 is the reserved polygon sentinel; validity is “ > -1.0 ”, never “ ≥ 0 ”.

Several of these affect *training*: the data and optimization the existing checkpoints were trained under depend on them, so changing one means retraining. That is the thread running through this whole codebase — it is a working, trained system, and its non-obvious choices are load-bearing.

This guide was generated from the project's own source and documentation.

Edit `chapters/*.tex` and rebuild with `build.sh` to keep it current.